

数字电路设计—DDCA 第 6 章详细学习笔记（更新版）

基于 *Digital Design and Computer Architecture: ARM® Edition*

2026 年 6 月 15 日

摘要

本文是 DDCA (Digital Design and Computer Architecture: ARM® Edition) 第 6 章的完整学习笔记，覆盖 260 页课件内容。本章系统地介绍了 ARM 体系结构的核心概念，从体系结构与微体系结构的区别出发，逐步深入到汇编语言、机器语言、编程基础、寻址模式等主题。笔记以中文为主，英文术语括号标注，所有要点均标注了对应的课件页码 [p.X]，适合作为期末考试复习资料。

目录

1	引言与体系结构概述	2
1.1	体系结构与微体系结构 [p.3]	2
1.2	指令 (Instructions) [p.4]	2
1.3	ARM 体系结构 [p.5–6]	2
1.4	体系结构设计原则 [p.7, 13, 15, 19]	2
1.5	ARM7TDMI 命名解析 [p.7]	3
2	汇编语言基础	3
2.1	基本指令：ADD 和 SUB [p.11–12]	3
2.2	多条指令组合 [p.14]	4
2.3	操作数位置 (Operand Location) [p.17]	4
2.4	ARM 寄存器集 [p.18–27]	4
2.4.1	ARM v7 寄存器 [p.20]	4
2.4.2	CPSR / APSR 状态寄存器 [p.21–22]	4
2.4.3	ARM v8 寄存器 [p.23–25]	5
2.5	寄存器使用约定 [p.26–27]	5
2.6	立即数操作 [p.29–33]	5

2.6.1	使用 MOV 生成小常数 [p.30–32]	5
2.6.2	使用 MOV + ORR 生成大常数 [p.33]	6
3	内存操作	6
3.1	字节寻址内存 [p.34–35]	6
3.2	读取内存: LDR [p.36–41]	6
3.3	写入内存: STR [p.42–45]	7
3.4	大端与小端 (Big-Endian & Little-Endian) [p.47–54]	7
3.5	IAR 内存管理 [p.55–65]	7
4	编程基础	8
4.1	编程构建块 [p.68–69]	8
4.2	数据处理指令 [p.70–89]	8
4.2.1	逻辑指令 [p.71–76]	8
4.2.2	移位指令 [p.77–84]	9
4.2.3	乘法指令 [p.85–89]	9
4.3	条件执行 (Conditional Execution) [p.90–101]	9
4.3.1	条件标志位 NZCV [p.92–93]	9
4.3.2	设置条件标志 [p.94–97]	10
4.3.3	条件助记符 [p.98–101]	10
4.4	分支 (Branches) [p.102–125]	11
4.4.1	分支类型 [p.103]	11
4.4.2	无条件分支 [p.105–106]	11
4.4.3	条件分支 [p.107–125]	11
4.5	数组 (Arrays) [p.130–137]	12
4.5.1	数组访问 [p.133–134]	12
4.5.2	使用 for 循环遍历数组 [p.135–136]	12
4.5.3	ASCII 码 [p.137–138]	13
4.6	函数调用 (Function Calls) [p.139–174]	13
4.6.1	ARM 函数调用约定 [p.141–143]	13
4.6.2	ARM 调用/返回指令 [p.143–146]	13
4.6.3	输入参数与返回值示例 [p.148–150]	14
4.6.4	栈 (The Stack) [p.151–153]	14
4.6.5	在栈上保存寄存器 [p.154–160]	14
4.6.6	PUSH/POP 便捷指令 [p.161–163]	15
4.6.7	叶子与非叶子函数 [p.165–168]	15
4.6.8	递归函数调用 [p.170–172]	15
4.6.9	函数调用总结 [p.174]	16

5 机器语言	16
5.1 指令编码原则 [p.175–177]	16
5.2 三种指令格式 [p.178]	16
5.3 数据处理指令格式 [p.179–210]	16
5.3.1 格式字段说明 [p.179–180]	16
5.3.2 带立即数的 DP 指令 [p.181–194]	17
5.3.3 带寄存器 Src2 的 DP 指令 [p.195–205]	17
5.3.4 寄存器移位寄存器 Src2 [p.203–204]	18
5.3.5 移位指令编码 [p.205–210]	18
5.4 内存指令格式 [p.211–226]	18
5.4.1 格式字段说明 [p.211]	18
5.4.2 寻址类型 (funct 字段) [p.212]	19
5.4.3 编码示例 [p.213–225]	19
5.5 分支指令格式 [p.227–234]	19
5.5.1 格式字段说明 [p.228]	19
5.5.2 分支目标地址计算 [p.229]	19
5.5.3 编码示例 [p.230–234]	20
5.6 解释机器码示例 [p.241–246]	20
5.6.1 示例 1: 0xE0475001 [p.241–244]	20
5.6.2 示例 2: 0xE5949010 [p.245–246]	20
5.7 存储程序的概念 [p.258–260]	20
6 寻址模式 (Addressing Modes) [p.247–257]	21
6.1 四种寻址模式	21
7 关键总结	22
7.1 ARM 指令集核心要点	22
7.2 常见考点速查	22

1 引言与体系结构概述

1.1 体系结构与微体系结构 [p.3]

- 体系结构 (Architecture): 程序员视角的计算机, 由指令和操作数位置定义
- 微体系结构 (Microarchitecture): 如何在硬件中实现体系结构 (第 7 章内容)
- 本章将抽象层次从数字逻辑向上跃升到指令集架构层面

1.2 指令 (Instructions) [p.4]

- 指令是计算机语言中的命令
- 汇编语言 (Assembly Language): 人类可读的指令格式
- 机器语言 (Machine Language): 计算机可读的格式 (0 和 1)
- 层次关系: 高级语言 → 汇编语言 → 机器语言 → 32 位地址

1.3 ARM 体系结构 [p.5–6]

- 由 Advanced RISC Machines (现 ARM Holdings) 于 1980 年代开发
- 每年销售近 100 亿颗 ARM 处理器
- 几乎所有的手机和平板都含有多颗 ARM 处理器
- 超过 75% 的人类使用含 ARM 处理器的产品
- 应用于服务器、相机、机器人、汽车、弹球机等
- 一旦学会一种体系结构, 学习其他的就更容易了

1.4 体系结构设计原则 [p.7, 13, 15, 19]

Hennessy 和 Patterson 提出的四大设计原则:

1. 规律性支持设计简洁性 (Regularity supports design simplicity) [p.13]
 - 一致的指令格式
 - 相同数量的操作数 (两个源操作数, 一个目标操作数)
 - 便于编码和硬件处理
2. 让常见情况更快 (Make the common case fast) [p.15–16]

- ARM 只包含简单、常用的指令
- 硬件解码和执行保持简单、小巧、快速
- 复杂指令（不常用）由多条简单指令组合完成
- ARM 是精简指令集计算机 (RISC)，指令少而简单
- x86 等是复杂指令集计算机 (CISC)
- RISC-V 是开放的 ISA，哈佛结构 (Cortex-M 系列)；冯诺依曼结构 (Cortex-A 系列)

3. 越小越快 (Smaller is faster) [p.19]

- ARM 只包含少量寄存器

4. 好的设计需要好的折中 (Good design demands good compromises) [p.177]

1.5 ARM7TDMI 命名解析 [p.7]

- T: Thumb (16 位指令集)
- D: 在线调试 (Debug)
- M: 双字相乘 (Multiplier)
- I: 硬件仿真 (ICE/EmbeddedICE)

2 汇编语言基础

2.1 基本指令：ADD 和 SUB [p.11–12]

- 加法: `ADD a, b, c` // C: `a = b + c;`
- 减法: `SUB a, b, c` // C: `a = b - c;`
- 助记符 (Mnemonic): 指示要执行的操作
- 源操作数 (Source operands): `b, c`
- 目标操作数 (Destination operand): `a`

2.2 多条指令组合 [p.14]

复杂代码由多条 ARM 指令组合完成：

```
// C: a = b + c - d;
// ARM:
ADD t, b, c    // t = b + c
SUB a, t, d    // a = t - d
```

2.3 操作数位置 (Operand Location) [p.17]

操作数在计算机中的物理位置有三种：

1. 寄存器 (Registers): 速度最快
2. 立即数 (Constants/Immediates): 值直接从指令中获取
3. 内存 (Memory): 容量大但慢

2.4 ARM 寄存器集 [p.18–27]

2.4.1 ARM v7 寄存器 [p.20]

ARM v7 有 16 个 32 位寄存器，因此称为“32 位体系结构”：

寄存器	用途
R0	参数/返回值/临时变量
R1–R3	参数/临时变量
R4–R11	保存变量 (Saved variables)
R12	临时变量
R13 (SP)	栈指针 (Stack Pointer)
R14 (LR)	链接寄存器 (Link Register)
R15 (PC)	程序计数器 (Program Counter)

2.4.2 CPSR / APSR 状态寄存器 [p.21–22]

当前程序状态寄存器 (CPSR)，在任何处理器模式下可访问：

- **N**: ALU 结果为负数 (Negative)
- **Z**: ALU 结果为零 (Zero)

- **C**: ALU 操作进位 (Carry)
- **V**: ALU 操作溢出 (oVerflow)
- **IT**: Thumb2 指令集中 If-Then 条件处理
- **I**: 屏蔽 IRQ 中断
- **F**: 屏蔽 FIQ 中断
- **T**: 核心是否处于 Thumb 状态
- **M[4:0]**: 处理器模式

APSR 是用户模式下的 CPSR 视图，包含 N、Z、C、V 四个标志位。SPSR 是备份程序状态寄存器。

2.4.3 ARM v8 寄存器 [p.23–25]

ARM v8 (AArch64) 有 31 个通用寄存器 (X0–X30)，使用 MSR/MRS 指令访问系统寄存器：

```
MSR TCR_EL1, X2    // 设置 TCR
MRS X0, SCTLR_EL1  // 读取系统控制寄存器
```

2.5 寄存器使用约定 [p.26–27]

- 寄存器命名：R 后跟数字，全大写，如”R0”
- 保存寄存器：R4–R11 存放变量
- 临时寄存器：R0–R3 和 R12，存放中间值

2.6 立即数操作 [p.29–33]

立即数 (Immediate)：值直接从指令中获取，用 # 前缀表示：

```
ADD R0, R0, #4    // R0 = R0 + 4
SUB R1, R0, #12    // R1 = R0 - 12
```

2.6.1 使用 MOV 生成小常数 [p.30–32]

```
MOV R0, #23        // int a = 23;
MOV R1, #0x45       // int b = 0x45;
```

约束：常数必须具有 <8 位精度（但可以通过 8 位立即数 +4 位循环移位来编码更宽范围）。

2.6.2 使用 MOV + ORR 生成大常数 [p.33]

对于超过 8 位精度的 32 位常数，分段构造：

```
// int a = 0x7EDC8765;
MOV R0, #0x7E000000
ORR R0, R0, #0xDC0000
ORR R0, R0, #0x8700
ORR R0, R0, #0x65
```

3 内存操作

3.1 字节寻址内存 [p.34–35]

- 寄存器太少无法存放所有数据 → 用内存存储
- 内存容量大但速度慢
- 常用变量仍然保存在寄存器中
- 每个数据字节有唯一地址
- 32 位字 = 4 字节，所以字地址以 4 递增
- 内存字地址 = 字序号 × 4 [p.46]

3.2 读取内存: LDR [p.36–41]

内存读称为加载 (Load)，助记符: **LDR** (Load Register):

```
LDR R0, [R1, #12]
// R0 = mem[R1 + 12]
// 地址 = 基址 (R1) + 偏移量 (12)
```

示例：读取内存地址 8 处的字到 R3 [p.39–40]:

```
MOV R2, #0
LDR R3, [R2, #8]
// 地址 = R2 + 8 = 8
// R3 = mem[8]
```


3.3 写入内存: STR [p.42–45]

内存写称为存储 (Store)，助记符: **STR** (Store Register):

示例: 将 R7 的值存入内存字 21 [p.43–45]:

```
// 内存地址 = 4 x 21 = 84 = 0x54
MOV R5, #0
STR R7, [R5, #0x54]
// 偏移量可用十进制或十六进制
```

3.4 大端与小端 (Big-Endian & Little-Endian) [p.47–54]

- 小端 (Little-endian): 字节编号从最低有效位开始
- 大端 (Big-endian): 字节编号从最高有效位开始
- 来源: Jonathan Swift 的《格列佛游记》——小端派从鸡蛋小端打破, 大端派从大端打破
- 实际影响: 只有在两个系统共享数据时才重要

经典例题 [p.50–52]: R2=8, R5=0x23456789, 问执行后 R7 的值:

```
STR R5, [R2, #0]
LDRB R7, [R2, #1]
// 大端: R7 = 0x00000045 (字节地址 9 = 0x45)
// 小端: R7 = 0x00000067 (字节地址 9 = 0x67)
```

地址布局对比:

大端				小端			
地址	8	9	A	B	B	A	9
8							
数据	23	45	67	89	23	45	67
89							
MSB ← LSB				LSB → MSB			

3.5 IAR 内存管理 [p.55–65]

- 使用 ICF 连接配置文件 (.icf) 分配和定位 ELF/DWARF 段
- 可编址存储空间、内存区域定义

- 数据定义指令：
 - DCD 32：定义 32 位数据
 - DS16 24：定义 16 字节存储空间
- ROOT vs NOROOT：NOROOT 表示如果没有符号引用则可以被链接器丢弃
- 查看内存：IAR 中 View -> Memory -> Memory1

4 编程基础

4.1 编程构建块 [p.68–69]

- 数据处理指令 (Data-processing Instructions)
- 条件执行 (Conditional Execution)
- 分支 (Branches)
- 高级结构：if/else、for 循环、while 循环、数组、函数调用

4.2 数据处理指令 [p.70–89]

4.2.1 逻辑指令 [p.71–76]

指令	全称	操作
AND	AND	按位与
ORR	OR	按位或
EOR	XOR	按位异或
BIC	Bit Clear	位清除 (AND NOT)
MVN	Move NOT	取反传送

用途示例 [p.73–76]：

- AND / BIC：掩码 (Masking) 位操作
 - $0xF234012F \text{ AND } 0x000000FF = 0x0000002F$ (保留最低字节)
 - $0xF234012F \text{ BIC } 0xFFFFFFFF00 = 0x0000002F$ (清除高字节)
- ORR：合并位域
 - $0xF2340000 \text{ ORR } 0x000012BC = 0xF23412BC$

4.2.2 移位指令 [p.77–84]

指令	全称	操作
LSL	Logical Shift Left	逻辑左移（填 0）
LSR	Logical Shift Right	逻辑右移（填 0）
ASR	Arithmetic Shift Right	算术右移（填符号位）
ROR	Rotate Right	循环右移

指令格式：

```
LSL R3, R2, #5      // R3 = R2 << 5
AND R5, R5, R9, LSL #3 // R5 = R5 & (R9 << 3)
```

用移位实现乘法 [p.84]：

- 左移 1 位 = 乘以 2：LSL R1, R0, #1
- 移位可以与 ADD/SUB 组合实现更复杂的乘法

4.2.3 乘法指令 [p.85–89]

- MUL：32 位乘法，结果低 32 位
- MLA：乘加 (Multiply-Accumulate)

```
MUL R0, R1, R2      // R0 = R1 * R2 （低 32 位）
MLA R0, R1, R2, R3   // R0 = R1 * R2 + R3
```

- 注意：乘法结果只保留低 32 位，可能溢出

4.3 条件执行 (Conditional Execution) [p.90–101]

4.3.1 条件标志位 NZCV [p.92–93]

ALU 设置四个条件标志位，保存在 CPSR 中：

标志	名称	含义
N	Negative	指令结果为负数
Z	Zero	指令结果为零
C	Carry	指令导致无符号进位
V	oVerflow	指令导致溢出

4.3.2 设置条件标志 [p.94–97]

方法 1: 使用比较指令 **CMP**:

`CMP R5, R6` // 执行 $R5 - R6$, 设置标志, 不保存结果

方法 2: 在指令助记符后加 **S** 后缀:

`ADDS R1, R2, R3` // $R1 = R2 + R3$, 同时设置标志位

`SUBS R0, R0, #1` // $R0 = R0 - 1$, 同时设置标志位

4.3.3 条件助记符 [p.98–101]

ARM 指令可以根据条件标志有条件地执行, 条件编码附加在指令助记符后:

后缀	全称	条件
EQ	Equal	$Z = 1$ (相等)
NE	Not Equal	$Z = 0$ (不等)
CS/HS	Carry Set / Higher or Same	$C = 1$ (无符号 $\geq$)
CC/LO	Carry Clear / Lower	$C = 0$ (无符号 $<$)
MI	Minus/Negative	$N = 1$ (负数)
PL	Plus/Positive or Zero	$N = 0$ (正数或零)
VS	oVerflow Set	$V = 1$ (溢出)
VC	oVerflow Clear	$V = 0$ (无溢出)
HI	Unsigned Higher	$C=1$ 且 $Z=0$ (无符号 $>$)
LS	Unsigned Lower or Same	$C=0$ 或 $Z=1$ (无符号 $\leq$)
GE	Signed Greater or Equal	$N=V$ (有符号 $\geq$)
LT	Signed Less Than	$N \neq V$ (有符号 $<$)
GT	Signed Greater Than	$Z=0$ 且 $N=V$ (有符号 $>$)
LE	Signed Less or Equal	$Z=1$ 或 $N \neq V$ (有符号 $\leq$)
AL	Always/Unconditional	总是执行

示例 [p.100–101]:

`CMP R5, R9` // $R5 - R9$, 设置标志

`SUBEQ R1, R2, R3` // 如果 $R5 == R9$ ($Z=1$), 执行

`ORRMI R4, R0, R9` // 如果结果为负 ($N=1$), 执行

若 $R5=17, R9=23$: `CMP` 执行 $17-23=-6$, 设置 $N=1, Z=0, C=0, V=0$; `SUBEQ` 不执行; `ORRMI` 执行。

4.4 分支 (Branches) [p.102–125]

4.4.1 分支类型 [p.103]

- **B** (Branch): 跳转到另一条指令
- **BL** (Branch and Link): 跳转并保存返回地址 (用于函数调用)
- 两种都可以是条件或无条件

4.4.2 无条件分支 [p.105–106]

```
MOV R2, #17           // R2 = 17
B TARGET              // 跳转到 TARGET
ORR R1, R1, #0x4       // 不执行
TARGET
SUB R1, R1, #78        // 执行
```

标签 (如 TARGET) 不能使用保留字 (如 ADD, ORR 等)。

4.4.3 条件分支 [p.107–125]

结合 CMP 和条件分支实现 if/else 逻辑:

if 语句 [p.115–117]:

```
// C: if (i == 0) a = b + c; else a = b - c;
CMP R0, #0           // 比较 i 和 0
BNE ELSE             // 如果不相等, 跳到 ELSE
ADD R1, R2, R3       // a = b + c
B END
ELSE
SUB R1, R2, R3       // a = b - c
END
```

while 循环 [p.121–122]:

```
// C: while (i < 10) { sum += i; i++; }
MOV R0, #0           // i = 0
MOV R1, #0           // sum = 0
FOR
CMP R0, #10          // i < 10?
BEQ DONE             // 如果 i==10, 退出
ADD R1, R1, R0       // sum += i
```

```

ADD R0, R0, #1    // i++
B FOR             // 继续循环
DONE

```

递减 for 循环（更高效） [p.127–129]:

```

// C: for (i=9; i!=0; i=i-1) sum = sum + i;
MOV R0, #9        // i = 9
MOV R1, #0        // sum = 0
FOR
ADD R1, R1, R0    // sum += i
SUBS R0, R0, #1   // i--, 并设置标志
BNE FOR          // 如果 i!=0, 继续

```

优点：每次迭代节省 2 条指令——递减循环变量同时比较 (SUBS)，只需 1 个分支而非 2 个。

4.5 数组 (Arrays) [p.130–137]

4.5.1 数组访问 [p.133–134]

```

// C: array[0] = array[0] * 8;
//      array[1] = array[1] * 8;
MOV R0, #0x60000000 // R0 = 基址
LDR R1, [R0]        // R1 = array[0]
LSL R1, R1, 3        // R1 = R1 * 8
STR R1, [R0]        // array[0] = R1
LDR R1, [R0, #4]    // R1 = array[1]
LSL R1, R1, 3        // R1 = R1 * 8
STR R1, [R0, #4]    // array[1] = R1

```

4.5.2 使用 for 循环遍历数组 [p.135–136]

```

// C: for (i=199; i>=0; i=i-1) array[i] = array[i] * 8;
MOV R0, 0x60000000
MOV R1, #199
FOR
LDR R2, [R0, R1, LSL #2] // R2 = array[i]
LSL R2, R2, #3           // R2 = R2 * 8
STR R2, [R0, R1, LSL #2] // array[i] = R2

```

```
SUBS R1, R1, #1          // i--, 设置标志
BPL FOR                  // if (i>=0) 继续
```

关键点: [R0, R1, LSL #2] 使用带缩放寄存器偏移的基址寻址。

4.5.3 ASCII 码 [p.137–138]

- 每个文本字符有唯一的字节值
- 例如: 'S' = 0x53, 'a' = 0x61, 'A' = 0x41
- 小写和大写字母相差 0x20 (32)

4.6 函数调用 (Function Calls) [p.139–174]

4.6.1 ARM 函数调用约定 [p.141–143]

- 调用者 (Caller):
 - 将参数传给被调用者 (R0–R3)
 - 跳转到被调用者 (BL)
- 被调用者 (Callee):
 - 执行函数功能
 - 将结果返回给调用者 (R0)
 - 返回到调用点 (MOV PC, LR)
 - 不能覆盖调用者需要的寄存器或内存

4.6.2 ARM 调用/返回指令 [p.143–146]

- 调用函数: BL function_label
 - 将返回地址 (PC+4) 保存到 LR
 - 跳转到目标函数
- 从函数返回: MOV PC, LR
 - 将 LR 中的返回地址复制到 PC
- 参数: R0–R3
- 返回值: R0

4.6.3 输入参数与返回值示例 [p.148–150]

```
// C: y = diffofsums(2, 3, 4, 5);
MAIN
    MOV R0, #2        // 参数 0 = 2
    MOV R1, #3        // 参数 1 = 3
    MOV R2, #4        // 参数 2 = 4
    MOV R3, #5        // 参数 3 = 5
    BL DIFFOFSUMS     // 调用函数
    MOV R4, R0        // y = 返回值

// int diffofsums(int f, int g, int h, int i)
DIFFOFSUMS
    ADD R8, R0, R1    // R8 = f + g
    ADD R9, R2, R3    // R9 = h + i
    SUB R4, R8, R9    // result = (f+g) - (h+i)
    MOV R0, R4        // 返回值放入 R0
    MOV PC, LR        // 返回
```

问题：函数覆盖了 R4, R8, R9——可能破坏调用者的数据。解决方案：使用栈。

4.6.4 栈 (The Stack) [p.151–153]

- 用于临时保存变量的内存区域
- 后进先出 (LIFO) 队列，像一叠盘子
- 向下增长：从高地址向低地址扩展
- **SP (R13)**：栈指针，指向栈顶
- 需要空间时扩展，不需要时收缩

4.6.5 在栈上保存寄存器 [p.154–160]

```
DIFFOFSUMS
    SUB SP, SP, #12    // 在栈上为 3 个寄存器分配空间
    STR R4, [SP, #8]   // 保存 R4
    STR R8, [SP, #4]   // 保存 R8
    STR R9, [SP]       // 保存 R9
    ADD R8, R0, R1     // R8 = f + g
```



```

ADD R9, R2, R3      // R9 = h + i
SUB R4, R8, R9      // result = (f+g) - (h+i)
MOV R0, R4          // 返回值放入 R0
LDR R9, [SP]        // 恢复 R9
LDR R8, [SP, #4]    // 恢复 R8
LDR R4, [SP, #8]    // 恢复 R4
ADD SP, SP, #12     // 释放栈空间
MOV PC, LR          // 返回

```

4.6.6 PUSH/POP 便捷指令 [p.161–163]

- PUSH {R4, R5, LR}: 将多个寄存器压栈
- POP {R4, R5, LR}: 从栈中弹出多个寄存器
- 自动更新 SP

4.6.7 叶子与非叶子函数 [p.165–168]

- 叶子函数 (Leaf): 不调用其他函数的函数。不需要保存 LR。
- 非叶子函数 (Nonleaf): 调用其他函数的函数。必须保存 LR!

非叶子函数示例 [p.167–168]: f1 调用 f2, f1 必须保存 LR、R4、R5; f2 是叶子函数, 只需保存 R4。

4.6.8 递归函数调用 [p.170–172]

阶乘 (factorial) 的 ARM 汇编实现:

FACTORIAL

```

STR R0, [SP, #-4]!   // 将R0压栈 (pre-decrement)
STR LR, [SP, #-4]!   // 将LR压栈
CMP R0, #2
BHS ELSE             // if (n >= 2) goto ELSE
MOV R0, #1           // return 1
ADD SP, SP, #8       // 恢复SP
MOV PC, LR           // 返回

```

ELSE

```

SUB R0, R0, #1       // n = n - 1
BL FACTORIAL         // 递归调用
LDR LR, [SP], #4     // 恢复LR (post-increment)

```

```

LDR R1, [SP], #4      // 恢复 R0 (原 n) 到 R1
MUL R0, R1, R0         // R0 = n * factorial(n-1)
MOV PC, LR            // 返回

```

注意: STR R0, [SP, #-4]! 中 ! 表示写回 (writeback), 即先 SP=SP-4, 再将 R0 存入新 SP 指向的位置。

4.6.9 函数调用总结 [p.174]

- 调用者: 参数放入 R0-R3 → 保存需要的寄存器 (LR, R0-R3, R8-R12) → BL CALLEE → 恢复寄存器 → 从 R0 取结果
- 被调用者: 保存可能被破坏的寄存器 (R4-R7) → 执行功能 → 结果放入 R0 → 恢复寄存器 → MOV PC, LR

5 机器语言

5.1 指令编码原则 [p.175–177]

- 32 位数据 → 32 位指令 (规律性)
- 但指令有不同的需求 (需要不同格式)
- 设计原则 4: 好的设计需要好的折中
- 多种指令格式提供灵活性, 但数量保持较少

5.2 三种指令格式 [p.178]

1. 数据处理 (Data-processing) 指令
2. 内存 (Memory) 指令
3. 分支 (Branch) 指令

5.3 数据处理指令格式 [p.179–210]

5.3.1 格式字段说明 [p.179–180]

- cond [31:28]: 4 位条件码
- op [27:26]: 00 = 数据处理指令

- **cmd [24:21]**: 4 位操作码 (ADD=4, SUB=2, ORR=12, AND=0, EOR=1, MOV=13, MVN=15, CMP=10 等)
- **S [20]**: 设置标志位 (1= 设置)
- **Rn [19:16]**: 第一源操作数寄存器
- **Rd [15:12]**: 目标寄存器
- **Src2 [11:0]**: 第二源操作数

5.3.2 带立即数的 DP 指令 [p.181–194]

I-bit = 1, Src2 = rot(4 位) + imm8(8 位)。32 位立即数 = imm8 循环右移 (rot x 2) 位。

示例: SUB R2, R3, #0xFF0 [p.190–194]

- cond=1110(14), op=00, I=1, cmd=0010(SUB)
- imm8=0xFF, rot=14 (ROR 28), Rd=2, Rn=3
- 机器码: 0xE2432EFF

5.3.3 带寄存器 Src2 的 DP 指令 [p.195–205]

I-bit = 0, bit4 = 0 表示寄存器 (未被移位), Src2 格式: shamt5(5 位) + sh(2 位) + 0 + Rm(4 位)。

移位类型编码 (sh) [p.205]:

sh	移位类型
00	LSL (逻辑左移)
01	LSR (逻辑右移)
10	ASR (算术右移)
11	ROR (循环右移)

示例 1: ADD R5, R6, R7 [p.198–200]

- cond=1110, op=00, cmd=0100(ADD), I=0, Rn=6, Rd=5, Rm=7
- shamt5=0, sh=00
- 机器码: 0xE0865007

示例 2: ORR R9, R5, R3, LSR #2 [p.202]

- cond=1110, op=00, cmd=1100(ORR), I=0, Rn=5, Rd=9, Rm=3

- shamt5=2, sh=01(LSR)
- 机器码: 0xE1859123

5.3.4 寄存器移位寄存器 Src2 [p.203–204]

bit4 = 1, Src2 格式: Rs(4 位) + 0 + sh(2 位) + 1 + Rm(4 位)。

示例: EOR R8, R9, R10, ROR R12 [p.204]

- cond=1110, op=00, cmd=0001(EOR), I=0, Rn=9, Rd=8, Rm=10, Rs=12
- sh=11(ROR)
- 机器码: 0xE0298C7A

5.3.5 移位指令编码 [p.205–210]

移位指令使用 cmd=1101(13), Src2 使用立即数移位寄存器格式。

示例: ROR R1, R2, #23 [p.206–207]

- cond=1110, op=00, cmd=1101, I=0, Rn=0, Rd=1, Rm=2
- shamt5=23, sh=11(ROR)
- 机器码: 0xE1A01BE2

5.4 内存指令格式 [p.211–226]

5.4.1 格式字段说明 [p.211]

- **op** [27:26]: 01 = 内存指令
- **funct** [25:20] + [11:4]: I(立即偏移), P(前变址), U(加), B(字节), W(写回), L(加载)
- **Rn** [19:16]: 基址寄存器
- **Rd** [15:12]: 目标 (LDR) 或源 (STR) 寄存器
- **Src2** [11:0]: 偏移量 (立即数或寄存器)

位	含义
I=0	立即数偏移；I=1: 寄存器偏移
P=1	前变址 (Pre-index)；P=0: 后变址 (Post-index)
U=1	加偏移；U=0: 减偏移
B=1	字节操作；B=0: 字操作
W=1	写回基址寄存器
L=1	加载 (LDR)；L=0: 存储 (STR)

5.4.2 寻址类型 (funct 字段) [p.212]

5.4.3 编码示例 [p.213–225]

LDR R4, [R0, #4] (前变址立即偏移) [p.213–215]: 机器码 0xE5904004

LDR R8, [R2], #10 (后变址立即偏移) [p.216–218]: 操作: $R8 = \text{mem}[R2]$; $R2 = R2 + 10$; 机器码 0xE492800A

LDRB R5, [R3, #-2] (字节加载) [p.219–221]: 机器码 0xE5535002

STR R11, [R5], #-26 (后变址减偏移) [p.222–223]: 操作: $\text{mem}[R5] = R11$; $R5 = R5 - 26$

LDR R3, [R4, R5] (寄存器偏移) [p.224]: 机器码 0xE7943005

STR R9, [R1, R3, LSL #2] (缩放寄存器偏移) [p.225]: 操作: $\text{mem}[R1 + (R3 \ll 2)] = R9$; 机器码 0xE7819103

5.5 分支指令格式 [p.227–234]

5.5.1 格式字段说明 [p.228]

- **op** [27:26]: 10 = 分支指令
- **imm24** [23:0]: 24 位有符号立即数
- **L** [24]: L=1 为 BL (带链接), L=0 为 B

5.5.2 分支目标地址计算 [p.229]

$$\text{BTA} = (\text{PC} + 8) + (\text{SignExtended}(\text{imm24}) \times 4)$$

imm24 表示距离 PC+8 的字数 (每条指令 4 字节)。

5.5.3 编码示例 [p.230–234]

示例 1（正向分支）[p.230–232]：PC=0xA0, PC+8=0xA8, 目标 THERE 在 0xB4。BTA 与 PC+8 相距 3 条指令, imm24=3。条件为 LT(1011), 所以 BLT THERE 编码为 0xBA000003。

示例 2（反向分支）[p.233–234]：PC=0x8050, PC+8=0x8058, 目标 TEST 在 0x8040。BTA 与 PC+8 相距-6 条指令, imm24=-6（24 位有符号）。

5.6 解释机器码示例 [p.241–246]

5.6.1 示例 1: 0xE0475001 [p.241–244]

- op=00 → 数据处理指令
- I=0 → Src2 是寄存器; bit4=0 → 未移位寄存器
- cmd=0010 (2) → SUB
- Rn=7, Rd=5, Rm=1, shamt5=0, sh=0
- 指令: SUB R5, R7, R1

5.6.2 示例 2: 0xE5949010 [p.245–246]

- op=01 → 内存指令
- funct: B=0(WORD), L=1(LDR), P=1(offset), W=0, I=0(imm), U=1(add)
- Rn=4, Rd=9, imm12=16
- 指令: LDR R9, [R4, #16]

5.7 存储程序的概念 [p.258–260]

- 32 位指令和数据存储在内存中
- 两个应用程序之间的唯一区别是指令序列
- 运行新程序: 无需重新布线, 只需将新程序存入内存
- 程序执行: 处理器按顺序从内存取指并执行
- PC (程序计数器) 跟踪当前指令地址

存储程序示例 [p.259]:

地 址	指 令 (机 器 码)
00000000	E3A01064 (MOV R1, #100)
00000004	E3A02045 (MOV R2, #69)
00000008	E2813002 (ADD R3, R1, R2)
0000000C	E5913000 (STR R3, [R1])

6 寻址模式 (Addressing Modes) [p.247–257]

6.1 四种寻址模式

1. 寄存器寻址 (Register Addressing) [p.249–250]

- 源和目标操作数在寄存器中
- 用于数据处理指令
- 三种子模式：
 - 纯寄存器：ADD R0, R2, R7
 - 立即数移位寄存器：ORR R5, R1, R3, LSL #1
 - 寄存器移位寄存器：SUB R12, R9, R0, ASR R1

2. 立即数寻址 (Immediate Addressing) [p.252]

- 源操作数在寄存器和立即数中
- 示例：ADD R9, R1, #14
- 使用数据处理格式，I=1
- 立即数编码为 8 位 imm8 + 4 位 rot
- 32 位立即数 = imm8 ROR (rot × 2)

3. 基址寻址 (Base Addressing) [p.254–255]

- 操作数地址 = 基址寄存器 + 偏移量
- 偏移量类型：
 - 12 位立即数：LDR R0, [R8, #-11]
 - 寄存器：LDR R1, [R7, R9]
 - 立即数移位寄存器：STR R5, [R3, R2, LSL #4]

4. PC 相对寻址 (PC-Relative Addressing) [p.257]

- 用于分支指令

- 操作数为 PC 和 24 位有符号立即数
- 新 PC 相对于旧 PC
- $BTA = (PC+8) + \text{SignExtended}(\text{imm24}) \times 4$

7 关键总结

7.1 ARM 指令集核心要点

1. 四大设计原则：规律性、常见情况快、越小越快、好的折中
2. 16 个 32 位寄存器：R0–R15，其中 SP=R13, LR=R14, PC=R15
3. 三条核心指令：ADD(加)、SUB(减)、MOV(传送)
4. 内存操作：LDR(加载)、STR(存储)，偏移量可立即数/寄存器/缩放寄存器
5. 条件执行：15 种条件码 + CMP/S 后缀设置 NZCV 标志
6. 分支：B(无条件)、BL(函数调用)、条件分支 (BEQ/BNE 等)
7. 函数调用：BL 保存返回地址到 LR，MOV PC, LR 返回，R0–R3 传参，R0 返回值
8. 栈：向下增长，SP 指向栈顶，PUSH/POP 管理
9. 三种指令格式：数据处理 (op=00)、内存 (op=01)、分支 (op=10)
10. 四种寻址模式：寄存器、立即数、基址、PC 相对

7.2 常见考点速查

- 大端/小端：STR 后 LDRB 不同字节，大端得高字节，小端得低字节
- MOV 立即数限制：8 位精度限制，大常数需 MOV+ORR 分段构造
- 递减循环优化：SUBS 设置标志 +BNE，每次省 2 条指令
- PC+8 计算：分支的 $BTA = (PC+8) + \text{imm24} \times 4$
- 寄存器约定：R0–R3 参数，R4–R11 保存，R0 返回值
- 非叶子函数：必须保存 LR 到栈，否则递归/嵌套调用会丢失返回地址
- CMP vs SUBS：CMP 不保存结果只设标志；带 S 指令设标志且保存结果
- 写回 (!)：前变址写回更新基址寄存器，后变址总是写回